

오늘의 작업기록 — 2026년 7월 23일

복기지 채점기 교체 — 위치 비교에서 LCS로

(어제 남긴 결함 하나를 고치며 동적계획법을 처음 만난 날)

[문서 규칙] 발체 없이 풀코드·풀출력으로 기재한다. 교체 전 코드와 교체 후 코드를 둘 다 전문으로 심는다 — 무엇이 왜 바뀌었는지 이 파일만으로 대조 가능하도록.

한눈에

항목	내용
고친 것	복기지 코드 재현 채점기 — 위치(index) 비교 → LCS(최장 공통 부분수열) 비교
결함	중간에 한 줄만 빠뜨려도 아래가 통째로 밀려 전부 오답. 95% 맞게 쳐도 20%가 틀 수 있음
★ 새 개념	LCS + 동적계획법(DP) — 테이블, 점화식, 역추적. git diff가 쓰는 바로 그 방식
지표 재설계	일치율(단일 숫자) → 재현율 + 빠뜨림 + 군더더기(셋으로 분해)
왜 분해하나	빠뜨림 = 기억의 구멍 / 군더더기 = 불필요한 습관 — 처방이 다르다
검증	시나리오 6종 node 테스트 — 중간 1줄 빠뜨림 ~40% → 92%, 빈 줄 추가 폭망 → 100%(군더더기 1)
배포	로컬 15127 bytes / lcsDiff 2, Pages에서 받은 것도 2 = 라이브 반영
복잡도	시간·공간 O(n×m) — 40줄 대조면 1600칸, 브라우저에서 즉시

어제 복기지를 만들면서 "일단 되게" 짜둔 비교 로직이 하나 있었다. 그게 학습 도구로는 못 쓸 물건이라는 걸 알고 있었고, 오늘 그걸 고쳤다. 결과적으로 동적계획법이라는 큰 개념을 처음 실물로 만난 날이 됐다 — 남이 설명해주는 예제가 아니라, 내 도구의 결함을 고치려다 필요해서 배운 것.

0. 어제 남긴 결함 — 위치 비교가 왜 못 쓰는 물건인가

어제 만든 채점기는 이렇게 동작했다: 원본 1번 줄 ↔ 내가 친 1번 줄, 원본 2번 줄 ↔ 내가 친 2번 줄을 차례로 맞대본다. 같으면 ✓, 다르면 ✕.

이게 왜 문제냐 — 구체적으로 보자. 원본이 이렇다고 하자:

```
1 public class Main {
2     public static void main(String[] a){
3         ServerSocket s = null;
4         int port = 6379;
5         try {
6             s = new ServerSocket(port);
7             s.setReuseAddress(true);
8             c = s.accept();
9         } catch (IOException e) {
```

내가 7번 줄 하나만 빠뜨리고 나머지를 완벽하게 쳤다고 하자. 위치 비교는 이렇게 판정한다:

위치	원본	내가 친 것	위치 비교 판정
1~6	(동일)	(동일)	✓
7	s.setReuseAddress(true);	c = s.accept();	✕
8	c = s.accept();	} catch (IOException e) {	✕
9	} catch (IOException e) {	(그 다음 줄)	✕
...	...	...	끝까지 전부 ✕

한 줄 빠뜨렸을 뿐인데 그 아래가 전부 밀려서 오답이 된다. 13줄짜리에서 7번을 빠뜨리면 6/13 = 46%, 더 긴 코드에서 앞쪽을 빠뜨리면 20%도 나온다. 실제로는 거의 다 맞게 쳤는데.

빈 줄 하나를 더 넣어도 똑같다. 오히려 더 혼하다 — 코드 칠 때 문단 나누려고 엔터 한 번 더 치는 건 자연스러운 습관이니까.

학습 도구에서 이게 왜 치명적인가: 점수가 실력을 반영 못 하면 피드백이 거짓말이 된다. 92% 실력인데 46%가 뜨면 ① 의욕이 꺾이고 ② "어디가 틀렸나"를 화면에서 못 찾는다 (다 빨간색이라). 도구를 안 쓰게 되는 가장 빠른 길이다.

1. ★ LCS — 개념 전체 (오늘의 새 개념)

git이 diff 를 보여줄 때 쓰는 방식이 정확히 이 문제를 푼다. 핵심 아이디어:

줄을 위치로 맞대지 말고, "양쪽에 공통으로 등장하는 가장 긴 줄 순서"를 먼저 찾아라. 그걸 기준으로 삼으면, 나머지는 자연스럽게 "원본에만 있는 줄(빠뜨림)"과 "내 답에만 있는 줄(군더더기)"로 갈린다.

그 "가장 긴 공통 순서"가 **LCS(Longest Common Subsequence, 최장 공통 부분수열)**다.

부분수열(subsequence)이란

연속일 필요가 없다. 순서만 지키면 된다.

- **A B C D**의 부분수열: **A C D** (O — B를 건너뛴), **B D** (O), **D A** (X — 순서가 바뀜)
- **부분문자열(substring)**은 연속이어야 하지만, **부분수열**은 **띄엄띄엄 골라도 된다**. 이 차이가 핵심이다 — 중간에 빠뜨린 줄이 있어도 앞뒤가 이어지는 이유가 여기 있다.

왜 이게 diff를 만드나

원본 **A B C D**, 내 답 **A C D** 라면 LCS는 **A C D** (길이 3)다. 그림: - LCS에 포함된 **A**, **C**, **D** → **맞은 줄** - 원본에만 있고 LCS 밖인 **B** → **빠뜨림** - 내 답에만 있고 LCS 밖인 줄 → **군더더기**(이 예엔 없음)

한 줄 빠뜨림이 딱 그 한 줄만 오답으로 잡힌다. 아래가 안 밀린다.

동적계획법(DP)으로 LCS 길이 구하기

모든 조합을 다 시도하면 경우의 수가 지수적으로 폭발한다. DP는 **작은 문제의 답을 표에 저장해두고 재사용**해서 이걸 다항 시간으로 만든다.

정의: **dp[i][j]** = "a의 i번째부터 끝까지"와 "b의 j번째부터 끝까지"의 LCS 길이.

점화식 (이게 전부다):

```
a[i] == b[j] 이면    dp[i][j] = dp[i+1][j+1] + 1
a[i] != b[j] 이면    dp[i][j] = max( dp[i+1][j], dp[i][j+1] )
```

말로 풀면: - **두 줄이 같으면** — 이 줄은 공통이니 챙기고(+1), 나머지는 양쪽 다 한 칸씩 밀린 문제의 답(**dp[i+1][j+1]**)을 가져온다. - **다르면** — 둘 중 하나를 버려야 한다. **a쪽 줄을 버린 경우**(**dp[i+1][j]**)와 **b쪽 줄을 버린 경우**(**dp[i][j+1]**) 중 **더 긴 쪽**을 택한다.

경계: **i**가 끝을 넘거나 **j**가 끝을 넘으면 남은 게 없으니 **0**. (배열을 **(n+1) × (m+1)** 크기로 잡아 자동으로 0이 되게 한다.)

왜 뒤에서부터 채우나: **dp[i][j]**는 **dp[i+1][j+1]**, **dp[i+1][j]**, **dp[i][j+1]** — 전부 **자기보다 오른쪽·아래** 칸만 참조한다. 그러니 오른쪽·아래부터 채워야 필요한 값이 이미 준비돼 있다.

실제 표로 따라가기 (a = A B C D, b = A C D)

dp는 5×4 크기. 마지막 행/열은 전부 0(경계).

	j=0 (A)	j=1 (C)	j=2 (D)	j=3 (경계)
i=0 (A)	3	2	1	0
i=1 (B)	2	2	1	0
i=2 (C)	2	2	1	0
i=3 (D)	1	1	1	0
i=4 (경계)	0	0	0	0

몇 칸만 직접 확인해보자: - **dp[3][2]** : a[3]=**D**, b[2]=**D** → **같다** → **dp[4][3] + 1 = 0 + 1 = 1** ✓ - **dp[2][1]** : a[2]=**C**, b[1]=**C** → **같다** → **dp[3][2] + 1 = 1 + 1 = 2** ✓ - **dp[1][1]** : a[1]=**B**, b[1]=**C** → **다르다** → **max(dp[2][1]=2, dp[1][2]=1) = 2** ✓ - **dp[0][0]** : a[0]=**A**, b[0]=**A** → **같다** → **dp[1][1] + 1 = 2 + 1 = 3** ✓

dp[0][0] = 3 = LCS 길이 3 (A C D). 맞다.

역추적(backtracking) — 표에서 diff 뽑아내기

길이만 알아선 화면에 못 보여준다. **표를 앞에서부터 되짚어 가며** 각 줄이 어디 속하는지 판정한다. **i=0, j=0**에서 출발:

1. **a[i] == b[j]** → 공통 줄. **ok**로 기록하고 **i++, j++** (둘 다 전진)
2. **다르고 dp[i+1][j] >= dp[i][j+1]** → a쪽을 버리는 게 이득. **miss(빠뜨림)**로 기록하고 **i++**
3. **다르고 그 반대** → b쪽을 버리는 게 이득. **extra(군더더기)**로 기록하고 **j++**
4. 한쪽이 먼저 끝나면 — a가 남았으면 전부 **miss**, b가 남았으면 전부 **extra**

위 예제(a= **A B C D**, b= **A C D**)로 돌려보면:

단계	비교	판정	다음
i=0,j=0	A == A	ok	i=1,j=1
i=1,j=1	B != C , dp[2][1]=2 ≥ dp[1][2]=1	miss(B)	i=2
i=2,j=1	C == C	ok	i=3,j=2
i=3,j=2	D == D	ok	i=4,j=3 (끝)

결과: **맞음 3, 빠뜨림 1(B), 군더더기 0**. 재현율 3/4 = **75%**. 위치 비교였다면 B 이후가 다 밀려서 **A**만 맞고 25%가 나왔을 것이다.

동점일 때(>=) miss를 택하는 이유: 어느 쪽을 택해도 LCS 길이는 같다. 다만 규칙을 하나로 고정해야 같은 입력에 항상 같은 화면이 나온다. "원본을 먼저 소비한다"로 정해두면 빠뜨린 줄이 위쪽에 모여 읽기도 낫다.

복잡도

- 시간·공간 모두 $O(n \times m)$ (n=원본 줄 수, m=내 답 줄 수)
- 40줄짜리 코드를 40줄로 재현하면 `41 × 41 = 1681` 칸. 브라우저에서 눈 깜짝할 새다.
- 저장은 `Uint16Array` 를 썼다 — 일반 배열보다 메모리가 훨씬 작고 빠르다. 줄 수가 65535를 넘을 일은 없으니 16비트로 충분하다.

2. 교체 전 코드 (풀코드) — 무엇을 버렸나

```
function compareCode(){
  if(!curCode){ alert('코드를 선택해라'); return; }
  clearInterval(tInt);
  const exp=curCode.code.split('\n').map(s=>s.replace(/\s+$/, ''));
  const got=document.getElementById('codeInput').value.split('\n').map(s=>s.replace(/\s+$/, ''));
  const n=Math.max(exp.length, got.length);
  let eq=0;
  const tbl=document.createElement('table'); tbl.className='diff';
  for(let i=0; i<n; i++){
    const e=exp[i]??'', g=got[i]??'', same=(e===g);
    if(same) eq++;
    const tr=document.createElement('tr'); tr.className=same?'ok':'bad';
    const td1=document.createElement('td'); td1.className='no'; td1.textContent=i+1;
    const td2=document.createElement('td'); td2.className='st'; td2.textContent=same?'✓':'x';
    const td3=document.createElement('td');
    td3.textContent = same ? e : (g||'(빈 줄)'+'\n→ '+e||'(원본엔 없음)');
    tr.append(td1,td2,td3); tbl.appendChild(tr);
  }
  const pct=Math.round(eq/n*100);
  const out=document.getElementById('diffOut'); out.innerHTML='';
  const sc=document.createElement('div');
  sc.className='score '+(pct>=90?'hi':pct>=60?'md':'lo');
  sc.textContent=`일치를 ${pct}% (${eq}/${n}줄) · ${document.getElementById('timer').textContent}`;
  out.append(sc, tbl);
  lastCodeResult=`${curCode.title} - 일치율 ${pct}% (${eq}/${n}줄), ${document.getElementById('timer').textContent},
${document.getElementById('blind').checked?'독타이핑':'보고치기'}`;
}
```

문제의 한 줄:

```
const e=exp[i]??'', g=got[i]??'', same=(e===g);
```

같은 인덱스 `i` 끼리만 비교한다. 밀림에 대한 방어가 전혀 없다.

부수적 문제도 하나 — 분모가 `Math.max(exp.length, got.length)` 다. 내가 쓸데없이 길게 치면 분모가 커져서 점수가 깎인다. 지표로서 의미가 흐릿하다.

3. 교체 후 코드 (풀코드)

추가한 CSS

```
tr.ok td.st { color:var(--ok); }
tr.miss td.st { color:var(--bad); } tr.miss { background:rgba(224,106,90,.09); }
tr.extra td.st { color:var(--mid); } tr.extra { background:rgba(224,182,74,.09); }
.tag { font-size:11px; opacity:.75; }
```

빠뜨림은 빨강, 군더더기는 노랑으로 색을 갈랐다 — 화면만 봐도 어느 종류의 실수인지 구분되도록. (교체 전엔 `tr.bad` 하나로 전부 빨강이었다.)

LCS 함수 + 새 채점 함수

```

/* LCS 정렬 비교 - git diff와 같은 방식.
한 줄을 빠뜨리거나 더 넣어도 그 줄만 잡히고, 나머지는 제자리에서 대조된다. */
function lcsDiff(a,b){
  const n=a.length,m=b.length;
  const dp=Array.from({length:n+1},()=>new Uint16Array(m+1));
  for(let i=n-1;i>=0;i--){
    for(let j=m-1;j>=0;j--){
      dp[i][j] = a[i]===b[j] ? dp[i+1][j+1]+1 : Math.max(dp[i+1][j],dp[i][j+1]);
    }
  }
  const out=[]; let i=0,j=0;
  while(i<n&&j<m){
    if(a[i]===b[j]){ out.push({t:'ok',line:a[i],ei:i}); i++; j++; }
    else if(dp[i+1][j]>=dp[i][j+1]){ out.push({t:'miss',line:a[i],ei:i}); i++; }
    else { out.push({t:'extra',line:b[j],ei:-1}); j++; }
  }
  while(i<n){ out.push({t:'miss',line:a[i],ei:i}); i++; }
  while(j<m){ out.push({t:'extra',line:b[j],ei:-1}); j++; }
  return out;
}

function compareCode(){
  if(!curCode){ alert('코드를 선택해라'); return; }
  clearInterval(tInt);
  const exp=curCode.code.split('\n').map(s=>s.replace(/\s+$/, ''));
  const got=document.getElementById('codeInput').value.split('\n').map(s=>s.replace(/\s+$/, ''));
  const rows=lcsDiff(exp,got);
  let ok=0,miss=0,extra=0;
  const tbl=document.createElement('table'); tbl.className='diff';
  rows.forEach(r=>{
    if(r.t==='ok') ok++; else if(r.t==='miss') miss++; else extra++;
    const tr=document.createElement('tr'); tr.className=r.t;
    const td1=document.createElement('td'); td1.className='no';
    td1.textContent = r.ei>=0 ? (r.ei+1) : '';
    const td2=document.createElement('td'); td2.className='st';
    td2.textContent = r.t==='ok' ? '✓' : r.t==='miss' ? '-' : '+';
    const td3=document.createElement('td');
    if(r.t==='ok') td3.textContent=r.line;
    else {
      const tag=document.createElement('span'); tag.className='tag';
      tag.textContent = r.t==='miss' ? '[빠뜨림]' : '[군더더기]';
      td3.append(tag, document.createTextNode(r.line||'(빈 줄)'));
    }
    tr.append(td1,td2,td3); tbl.appendChild(tr);
  });
  const pct = exp.length ? Math.round(ok/exp.length*100) : 0;
  const out=document.getElementById('diffOut'); out.innerHTML='';
  const sc=document.createElement('div');
  sc.className='score '+(pct>=90?'hi':pct>=60?'md':'lo');
  const time=document.getElementById('timer').textContent;
  sc.textContent=`재현율 ${pct}% · 맞춤 ${ok}/${exp.length}줄 · 빠뜨림 ${miss} · 군더더기 ${extra} · ${time}`;
  out.append(sc,tbl);
  lastCodeResult=`${curCode.title} - 재현율 ${pct}% (맞음 ${ok}/${exp.length}, 빠뜨림 ${miss}, 군더더기 ${extra}), ${time},
${document.getElementById('blind').checked?'독타이핑':'보고치기'}`;
}

```

줄별 해설

```
const dp=Array.from({length:n+1},()=>new Uint16Array(m+1));
```

(n+1) × (m+1) 2차원 배열. **+1인 이유는 경계(0) 처리** — **dp[n][*]** 와 **dp[*][m]** 이 자동으로 0이라 조건문 없이 점화식이 성립한다. **Array.from** 에 함수를 넘겨야 하는 이유: **Array(n+1).fill(new Uint16Array(...))** 로 하면 **모든 행이 같은 배열을 가리켜** 전부 뭉개진다.

```

for(let i=n-1;i>=0;i--){
  for(let j=m-1;j>=0;j--){
    dp[i][j] = a[i]===b[j] ? dp[i+1][j+1]+1 : Math.max(dp[i+1][j],dp[i][j+1]);
  }
}

```

뒤에서부터 채운다(\$ 1의 이유). 점화식이 그대로 삼항 연산자 한 줄이다.

```
if(a[i]===b[j]){ out.push({t:'ok',line:a[i],ei:i}); i++; j++; }
```

ei 는 **원본에서의 줄 번호**(expected index). 화면 왼쪽에 원본 기준 번호를 찍기 위해 들고 다닌다. 군더더기는 원본에 없는 줄이라 **ei:-1** 이고, 화면엔 번호 대신 **.** 을 찍는다.

```
else if(dp[i+1][j]>=dp[i][j+1]){ out.push({t:'miss',line:a[i],ei:i}); i++; }
else { out.push({t:'extra',line:b[j],ei:-1}); j++; }
```

표를 보고 **어느 쪽을 버릴지** 결정하는 부분. 동점이면 miss 우선(\$ 1).

```
while(i<n){ ... miss ... }
while(j<m){ ... extra ... }
```

한쪽이 먼저 끝난 뒤의 **꼬리 처리**. 원본이 남으면 전부 빠뜨림(절반만 치고 만 경우), 내 답이 남으면 전부 군더더기.

```
const pct = exp.length ? Math.round(ok/exp.length*100) : 0;
```

분모가 **exp.length** (원본 줄 수)로 고정됐다. 교체 전의 **Math.max(...)** 와 달리, 내가 쓸데없이 길게 쳐도 재현율은 안 깎이고 대신 **군더더기 카운트**가 올라간다. 지표가 각자 한 가지 의미만 갖게 된 것.

4. 채점 지표 재설계 — 왜 셋으로 쪼갬나

교체 전: 일치율 46% (6/13줄) — 숫자 하나. 교체 후: 재현율 92% · 맞음 12/13줄 · 빠뜨림 1 · 군더더기 0 · 03:41

왜 쪼개는 게 나은가 — 진단이 갈리기 때문이다:

지표	의미	많으면 진단	처방
재현율	원본 줄 중 맞게 친 비율	낮으면 전반적 미숙	보고치기 모드로 먼저 익히기
빠뜨림	원본엔 있는데 안 친 줄	기억의 구멍 — 그 부분을 아예 모르거나 잊음	그 줄이 나온 로그 섹션 재독
군더더기	원본엔 없는데 친 줄	불필요한 습관 — 안 배운 걸 넣거나 빈 줄 남발	원본의 구조를 다시 보기

같은 "70점"이라도 **빠뜨림** 위주면 기억이 비어 있는 것이고, **군더더기** 위주면 기억은 있는데 형태가 어긋난 것이다. 처방이 완전히 다르다. 숫자 하나로는 이걸 못 가른다.

화면 표시도 셋으로 갈렸다: -  — 맞음(초록) -  **[빠뜨림]** — 원본에 있는데 안 친(빨강 배경) -  **[군더더기]** — 원본에 없는데 친(노랑 배경)

5. 검증 — 시나리오 6종

알고리즘을 바꿨으니 **실제로 의도대로 채점되는지** 확인해야 한다. 브라우저 없이 node로 LCS 부분만 떼어 테스트했다.

테스트 코드 (풀코드)

```
function lcsDiff(a,b){
    const n=a.length,m=b.length;
    const dp=Array.from({length:n+1},()=>new Uint16Array(m+1));
    for(let i=n-1;i>=0;i--){
        for(let j=m-1;j>=0;j--){
            dp[i][j] = a[i]===b[j] ? dp[i+1][j+1]+1 : Math.max(dp[i+1][j],dp[i][j+1]);
        }
    }
    const out=[]; let i=0,j=0;
    while(i<n&&j<m){
        if(a[i]===b[j]){ out.push({t:'ok',line:a[i],ei:i}); i++; j++; }
        else if(dp[i+1][j]>=dp[i][j+1]){ out.push({t:'miss',line:a[i],ei:i}); i++; }
        else { out.push({t:'extra',line:b[j],ei:-1}); j++; }
    }
    while(i<n){ out.push({t:'miss',line:a[i],ei:i}); i++; }
    while(j<m){ out.push({t:'extra',line:b[j],ei:-1}); j++; }
    return out;
}

function score(exp,got){
    const r=lcsDiff(exp,got);
    let ok=0,miss=0,ex=0;
    r.forEach(x=>x.t==='ok'?ok++:x.t==='miss'?miss++:ex++);
    return {pct:Math.round(ok/exp.length*100),ok,miss,ex,n:exp.length};
}

const exp=['public class Main {','    public static void main(String[] a){','        ServerSocket s = null;','        int port = 6379;','        try {','            s
= new ServerSocket(port);','            s.setReuseAddress(true);','            c = s.accept();','        } catch (IOException e) {'','        }
System.out.println(e);','    }','}'];

// 시나리오 1: 완벽
console.log('완벽: ', JSON.stringify(score(exp,exp.slice())));
// 시나리오 2: 중간 한 줄 빠뜨림 (구 방식이면 아래 전부 밀려 폭망하던 케이스)
let g2=exp.slice(); g2.splice(6,1);
console.log('중간 1줄 빠뜨림: ', JSON.stringify(score(exp,g2)));
// 시나리오 3: 빈 줄 하나 더 넣음
let g3=exp.slice(); g3.splice(4,0,'');
console.log('빈 줄 1개 추가: ', JSON.stringify(score(exp,g3)));
// 시나리오 4: 한 줄 오타
let g4=exp.slice(); g4[5]='        s = new ServerSocket(6379);';
console.log('1줄 오타: ', JSON.stringify(score(exp,g4)));
// 시나리오 5: 절반만 침
console.log('절반만 침: ', JSON.stringify(score(exp,exp.slice(0,6))));
// 시나리오 6: 빈 답
console.log('빈 답: ', JSON.stringify(score(exp,[''])));
```

실행 결과 (전문)

JS 문법 OK	
완벽:	{ "pct":100,"ok":13,"miss":0,"ex":0,"n":13 }
중간 1줄 빠뜨림:	{ "pct":92,"ok":12,"miss":1,"ex":0,"n":13 }
빈 줄 1개 추가:	{ "pct":100,"ok":13,"miss":0,"ex":1,"n":13 }
1줄 오타:	{ "pct":92,"ok":12,"miss":1,"ex":1,"n":13 }
절반만 침:	{ "pct":46,"ok":6,"miss":7,"ex":0,"n":13 }
빈 답:	{ "pct":0,"ok":0,"miss":13,"ex":1,"n":13 }

판정

시나리오	새 방식	구 방식이었다면	평가
완벽	100%	100%	동일 ✓
중간 1줄 빠뜨림	92% (빠뜨림 1)	아래가 다 밀려 ~46%	핵심 개선 ✓
빈 줄 하나 추가	100% (군더더기 1)	통째로 폭망	핵심 개선 ✓
1줄 오타	92% (빠뜨림 1·군더더기 1)	92%	동일하되 원인이 보임 ✓
절반만 침	46% (빠뜨림 7)	46%	동일 ✓
빈 답	0% (빠뜨림 13)	0%	동일 ✓

오타 케이스가 흥미롭다 — 한 줄을 틀리게 치면 "원본 줄을 빠뜨림 + 엉뚱한 줄을 군더더기로 추가"로 잡힌다(빠뜨림1·군더더기1). 점수는 구방식과 같은 92%지만, 화면엔 원본 줄과 내가 친 줄이 나란히 보여서 무엇을 잘못 쳤는지 즉시 확인된다.

6. 배포와 라이브 확인

review.html은 언더바가 없어서 어제 겪은 파일명 변형(\$ 어제 \$ 6) 함정에 안 걸린다. SFTP로 덮어쓰기.

로컬 검증:

```
root@ledger:~/backend-journey# ls -la review.html && grep -c "lcsDiff" review.html
-rw-r--r-- 1 root root 15127 Jul 22 11:57 review.html
2
```

- 15127 bytes — 교체 전 13828에서 커졌다(LCS 함수 + CSS 추가분).
- lcsDiff 2개 — 함수 정의 1 + compareCode 안의 호출 1. 새 버전이 맞다.

라이브(GitHub Pages) 검증:

```
root@ledger:~/backend-journey# sleep 70 && curl -s https://taewoo-won.github.io/backend-journey/review.html | grep -c lcsDiff
2
```

Pages에서 실제로 받아온 파일에도 lcsDiff 가 2개 = 크론 자동 push와 Pages 빌드가 다 끝났다는 뜻. 브라우저 없이 이것만으로 라이브 반영이 판정된다.

판정 도구로서의 grep -c : 7/14에 배운 것("ExceptionHandler" =4 vs "@ExceptionHandler" =3)의 재사용이다. 파일 크기만 보면 "뭔가 바뀌었다"까지만 알지만, 특정 식별자를 세면 "무엇으로 바뀌었다"까지 안다.

7. 로드맵 위치

- 복기지 v1 완성 — 어제 만든 것의 유일한 결함이 오늘 닫혔다. 이제 사지방 시간에 문제 복기 + 코드 재현을 제대로 돌릴 수 있다.
- 오늘 배운 LCS/DP는 로드맵 밖 소재지만 값이 크다 — 동적계획법은 알고리즘의 큰 기둥 하나이고, 8월 Layer 0(WAL) 이후 자료구조·알고리즘을 볼 때 다시 만난다. 도구를 고치려다 필요해서 배운 것이라 맥락이 붙어 있다.
- m101~m106 done, m107(월말 점검)만 남음. codecrafters-redis Stage 2(PING/PONG, RESP 프로토콜 첫 발)는 자유 슬롯.
- 밀린 복기 큐(14일+): 7/5 원자성 트레이스 + 문답집 ★15문항. 복기지가 완성됐으니 이제 미를 이유가 없다 — 다음 세션 첫 순서.
- 8월 WAL 이유 3개 유지: 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

8. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 위치(index) 비교가 학습 도구로 못 쓸 물건인 이유는? 13줄 중 7번째를 빠뜨리면 구방식은 몇 %가 나오고 왜 그런가?
 2. 부분수열(subsequence)과 부분문자열(substring)의 차이는? LCS가 "밀림"에 강한 이유가 이 차이와 어떻게 연결되나?
 3. ★ dp[i][j] 의 정의를 말로 쓰라. 그리고 점화식 두 갈래(갈을 때·다를 때)를 각각 이유와 함께 설명하라.
 4. DP 테이블을 왜 뒤에서부터 채우나? 각 칸이 어느 칸들에 의존하는지로 답하라.
 5. 배열을 (n+1) × (m+1) 로 잡은 이유는? Array(n+1).fill(new Uint16Array(m+1)) 로 만들면 무슨 일이 생기나?
 6. 역추적에서 ok/miss/extra를 가르는 세 조건은? 동점(>=)일 때 miss를 택하도록 고정한 이유는?
 7. LCS의 시간·공간 복잡도는? 40줄 코드를 40줄로 재현할 때 몇 칸을 계산하나?
 8. 채점 지표를 "일치율" 하나에서 셋으로 쪼갠 이유는? 빠뜨림이 많을 때와 군더더기가 많을 때의 진단·처방은 각각 무엇인가?
 9. 분모를 Math.max(exp.length, got.length) 에서 exp.length 로 바꾼 이유는? 내가 길게 치면 이제 어느 지표가 움직이나?
 0. 한 줄 오타가 새 방식에서 "빠뜨림 1 + 군더더기 1"로 잡히는 이유는? 점수는 구방식과 같은데 무엇이 나아졌나?
1. 브라우저 없이 라이브 반영을 판정한 방법은? 파일 크기만 보는 것과 grep -c 로 식별자를 세는 것의 차이는?

9. 회고

어제 복기지를 만들면서 비교 로직 하나를 "일단 되게" 짜줬다. 줄 번호를 맞대는 방식이었고, 그게 학습 도구로는 못 쓸 물건이라는 걸 그때도 알고 있었다 — 한 줄만 빠뜨려도 아래가 통째로 밀려 오답이 되니까. 실제로는 92% 맞게 쳤는데 46%가 뜨는 도구는, 점수가 실력을 반영 못 하는 도구다. 그런 도구는 피드백이 거짓말이고, 결국 안 쓰게 된다. 그래서 오늘 첫 일로 고쳤다.

고치는 과정에서 예상보다 큰 걸 만났다. LCS와 동적계획법. 이름은 들어봤지만 실제로 짜본 건 처음이다. 그런데 이번엔 "알고리즘 공부"로 만난 게 아니라 내 도구의 구체적 결함을 고치려다 필요해서 만났다. 이게 차이가 크다 — dp[i][j]가 뭘 의미하는지, 왜 뒤에서부터 채우는지, 왜 (n+1)×(m+1)인지가 전부 "내 문제를 풀기 위한 이유"로 붙어 있다. 표를 손으로 채워보며 dp[0][0]=3이 나오는 걸 확인한 순간이 오늘 제일 좋았다. 이게 포폴 슬로건("만들어서 안다")이 실제로 작동한 장면이다.

지표를 셋으로 쪼갠 것도 만족스럽다. 처음엔 그냥 일치율 하나면 되겠거니 했는데, 쓰다 보니 같은 70점이라도 원인이 완전히 다르다는 게 보였다. 빠뜨림이 많으면 기억이 비어 있는 거고, 군더더기가 많으면 기억은 있는데 형태가 어긋난 거다. 전자는 로그를 다시 읽어야 하고 후자는 구조를 다시 봐야 한다. 처방이 다르면 지표도 갈라야 한다 — 이걸 앞으로 다른 데서도 쓸 원칙 같다.

검증을 시나리오 6종으로 돌린 것도 남는다. 알고리즘을 바꿨으면 "잘 되는 것 같다"로 넘기면 안 된다. 완벽·빠뜨림·추가·오타·절반·빈 답을 다 넣어보고, 특히 구방식이 폭발하던 두 케이스가 실제로 정상화됐는지를 숫자로 확인했다. 92%와 100%가 찍히는 걸 보고서야 배포했다. 이걸 4연타 검증(7/14)에서 배운 습관 — 신규와 회귀를 함께 본다 — 의 다른 형태다.

이제 복기지에 남은 결함은 없다. 사지방 브라우저만 있으면 163문항을 인출하고 57개 코드를 빈 화면에 재현할 수 있다. 14일 밀린 복기 큐를 더 미를 이유가 사라졌으니, 다음 세션 첫 순서는 그거다.

한 줄: 복기지의 채점기를 위치 비교에서 LCS(동적계획법)로 교체 — 한 줄 밀림에 무너지던 점수가 92%/100%로 정상화됐고, 지표를 재현율·빠뜨림·군더더기 셋으로 쪼

